EXPERIMENT 7: Perceptron and Forward Propagation

1. Aim

To implement a single-layer perceptron using NumPy, train it on a binary loan-approval dataset, visualize its decision boundary, demonstrate why a single perceptron cannot solve the XOR problem, and compare step, sigmoid and ReLU activation functions.

2. Mathematical Formulae

Weighted sum

$$z = \sum_{i=1}^{n} w_i x_i + b$$

Step activation function

$$f(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$$

Perceptron learning rule

$$w_i^{new} = w_i^{old} + \eta(y - \hat{y})x_i$$

$$b^{new} = b^{old} + \eta(y - \hat{y})$$

Sigmoid activation

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

ReLU activation

$$ReLU(z) = \max(0, z)$$

Where x_i is an input, w_i is its weight, b is the bias, η is the learning rate, and $y - \hat{y}$ is the prediction error.

3. System Architecture



4. Algorithm / Procedure

1. Import NumPy and Matplotlib.
2. Generate a binary dataset representing loan applications.
3. Initialize weights and bias to zero.
4. Calculate the weighted sum and apply the step activation.
5. Update weights and bias using the perceptron learning rule.
6. Plot the decision boundary for the loan dataset.
7. Train the same perceptron on XOR data and observe its failure to classify all four points.
8. Compare step, sigmoid and ReLU outputs.

5. Python Program

```
import numpy as np

import matplotlib.pyplot as plt


np.random.seed(42)


# Generate synthetic loan data

n = 100

X = np.random.rand(n, 2)
```

```
# Labels: 1 = Approved, 0 = Rejected
y = ((X[:, 0] + X[:, 1]) > 1.0).astype(int)

def step(z):
    return (z >= 0).astype(int)

def sigmoid(z):
    return 1 / (1 + np.exp(-np.clip(z, -500, 500)))

def relu(z):
    return np.maximum(0, z)

# Train a single perceptron
weights = np.zeros(2)
bias = 0.0
learning_rate = 0.1
epochs = 30
errors_per_epoch = []

for epoch in range(epochs):
    errors = 0

    for xi, target in zip(X, y):
        z = np.dot(xi, weights) + bias
        prediction = int(step(z))
        error = target - prediction

        weights += learning_rate * error * xi
        bias += learning_rate * error
```

```
        if error != 0:
            errors += 1

    errors_per_epoch.append(errors)

# Evaluate the trained model
predictions = step(X @ weights + bias)
accuracy = np.mean(predictions == y)

print("Learned weights:", weights)
print("Learned bias:", round(bias, 4))
print("Training accuracy:", round(accuracy * 100, 2), "%")

# Plot decision boundary
plt.figure(figsize=(7, 5))

plt.scatter(
    X[y == 0, 0], X[y == 0, 1],
    label="Rejected", marker="x"
)
plt.scatter(
    X[y == 1, 0], X[y == 1, 1],
    label="Approved", marker="o"
)

x_line = np.linspace(0, 1, 100)

if abs(weights[1]) > 1e-8:
    y_line = -(weights[0] * x_line + bias) / weights[1]
    plt.plot(x_line, y_line, label="Decision Boundary")
```

```
plt.xlabel("Credit Score (scaled)")
plt.ylabel("Income-to-Debt Ratio (scaled)")
plt.title("Perceptron Loan Classification")
plt.legend()
plt.show()

# Plot errors per epoch
plt.plot(errors_per_epoch, marker="o")
plt.xlabel("Epoch")
plt.ylabel("Number of Misclassifications")
plt.title("Perceptron Training Errors")
plt.show()

# XOR dataset
X_xor = np.array([
  [0, 0],
  [0, 1],
  [1, 0],
  [1, 1]
])

y_xor = np.array([0, 1, 1, 0])

w_xor = np.zeros(2)
b_xor = 0.0

for epoch in range(20):
  for xi, target in zip(X_xor, y_xor):
    prediction = int(step(np.dot(xi, w_xor) + b_xor))
    error = target - prediction
```

```
        w_xor += 0.1 * error * xi
        b_xor += 0.1 * error

xor_predictions = step(X_xor @ w_xor + b_xor)

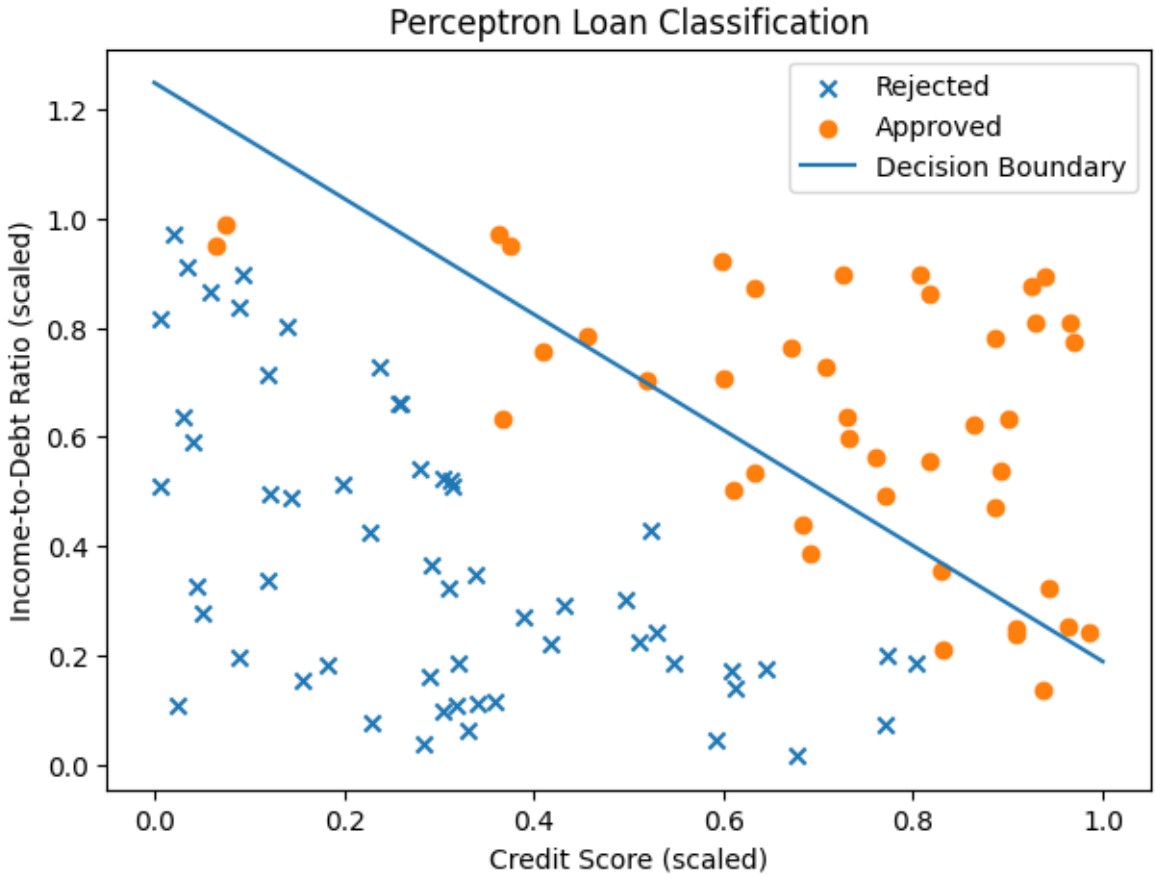
print("\nXOR actual labels:     ", y_xor)
print("XOR predicted labels:  ", xor_predictions)
print("XOR accuracy:", np.mean(xor_predictions == y_xor))

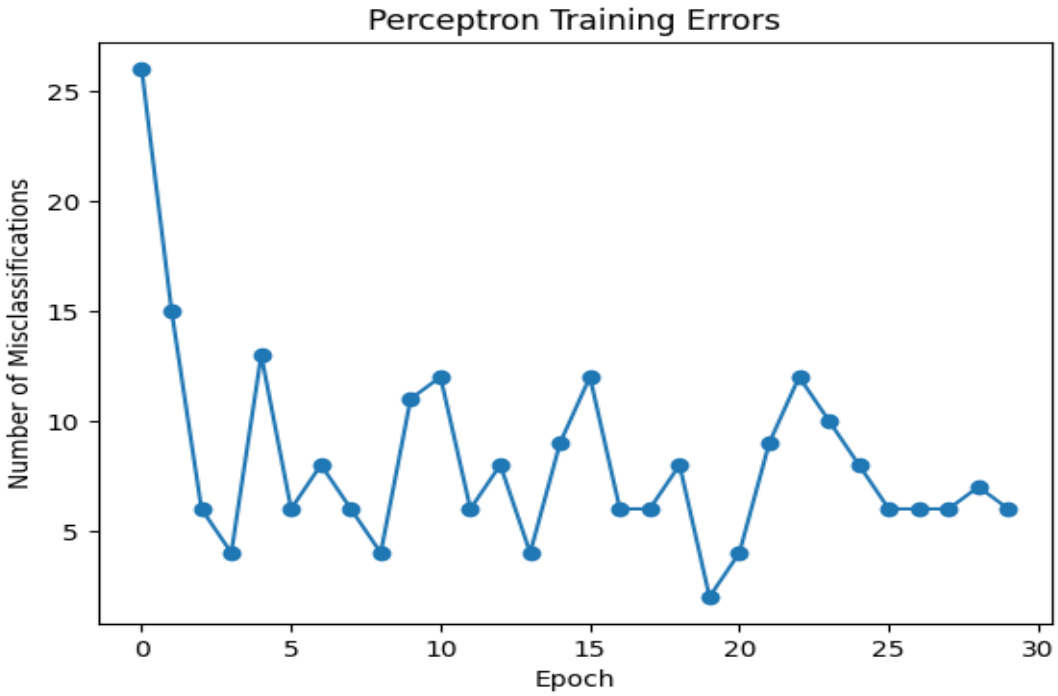
# Compare activation functions
z = np.linspace(-5, 5, 200)

plt.figure(figsize=(8, 5))
plt.plot(z, (z >= 0).astype(int), label="Step")
plt.plot(z, sigmoid(z), label="Sigmoid")
plt.plot(z, relu(z), label="ReLU")
plt.xlabel("Input z")
plt.ylabel("Activation Output")
plt.title("Activation Function Comparison")
plt.legend()
plt.grid(True)
plt.show()
```

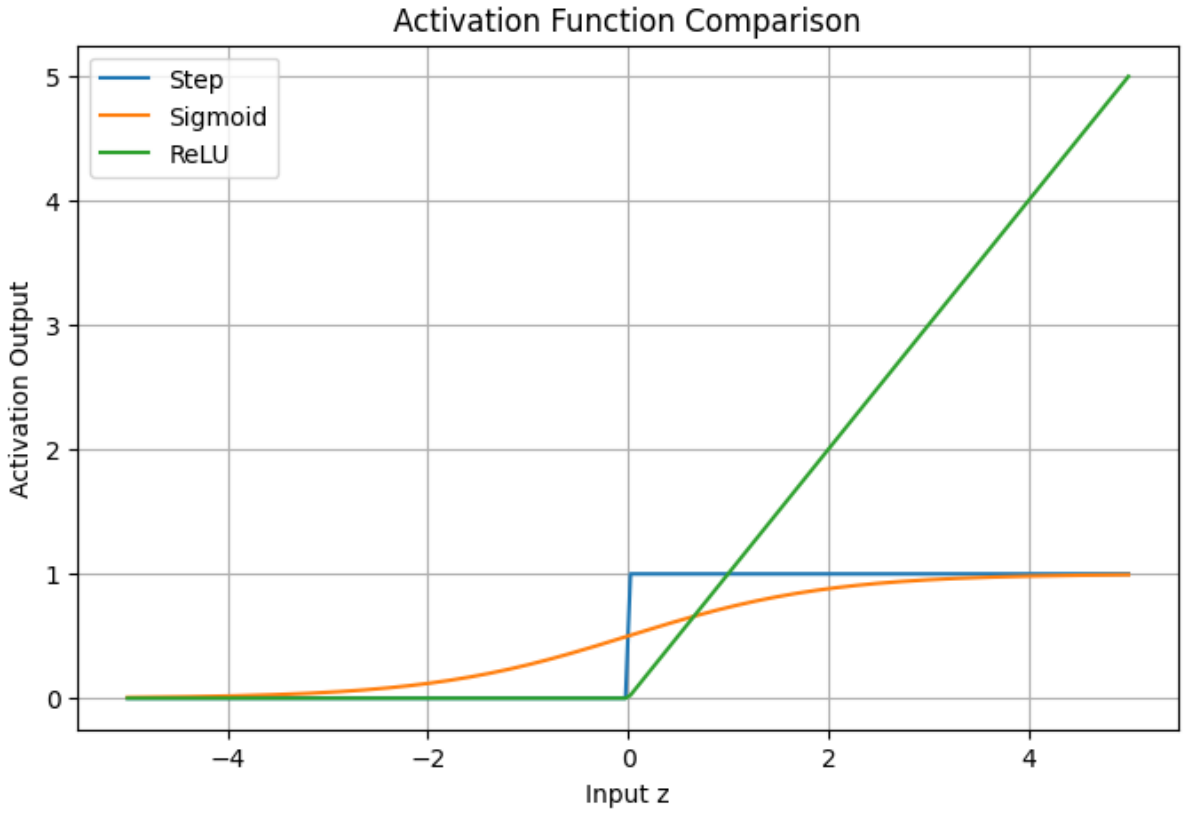
6. Output

```
Learned weights: [0.67873308 0.6410944 ]
Learned bias: -0.8
Training accuracy: 87.0 %
```





```
XOR actual labels:      [0 1 1 0]
XOR predicted labels:   [1 1 0 0]
XOR accuracy: 0.5
```



7. Result

A single-layer perceptron was implemented and trained using the perceptron learning rule. Its decision boundary was visualized, and the XOR experiment demonstrated the limitation of a single-layer perceptron in solving non-linearly separable problems.